

Carlos Guerrero

Professor Regan

Modeling and Simulation

25 June 2026

M2 Progress Documentation

Project Board

Title	***	Assignees	***	Status	≡ ↑ ***
1  production interruption implementation #10				To Do	
2  Direct request priority queue #14				To Do	
3  Working hours time handling #15				To Do	
4  Throughput and turnaround metric #16				To Do	
5  AdvEx flow #17				To Do	
6  Reship inspection #18				To Do	
7  Tech and station utilization #19				To Do	
8  Validation Checks #20				To Do	
9  M2 documentation #12				In progress	
10  Add basic structure #1				Done	
11  Data loading #3				Done	
12  Parse data after data loaded #4				Done	
13  Data analysis #13				Done	
14  Job priority per data #5				Done	
15  Repair outcomes per data #6				Done	
16  Arrival and Service Times #7				Done	
17  FIFO implementation #8				Done	
18  technician and station resource implementation #9				Done	
19  data collection implementation #11				Done	

The completed work includes the initial M2 commit of fixing repo structure and basic source code. Uploading the CSV data from our real-world department jobs and removing any company/customer information. It also includes parsing the CSV data, data analysis, priority

and outcome, arrival and service times, FIFO logic, resource matching, and basic SimPy run and metrics.

The items left to do are functions that will be implemented and completed in the next iterations of development. These include production interruptions, direct request priority, working-hours time handling, throughput and turn around metrics.

Implementation Evidence

I: CSV Data Summary - python src\print_csv_summary.py

```
PS C:\Users\carlo> cd "C:\Users\carlo\OneDrive\Documents\Modeling and Simulation\CS4632-Electronics-Repair"
PS C:\Users\carlo\OneDrive\Documents\Modeling and Simulation\CS4632-Electronics-Repair> python src\print_csv_summary.py
RMA parent jobs: 944
RMA unit jobs: 2366
Production issue jobs: 428

RMA priorities
ASAP: 342
Critical: 125
Normal: 477

RMA types
AdvEx: 105
Credit: 10
Repair & Return: 346
Reship: 3
Return - Credit: 362
Return - Replacement: 116
blank: 2

Unit outcomes
repaired: 838
rtv: 520
scrap: 181
unknown: 827

Unit capabilities
general: 1564
specialty: 802

Production locations
Advantech: 1
Belluno: 21
Benchmark: 117
Chihuahua: 237
Hantronix: 7
M2S: 4
Prime: 21
blank: 20
PS C:\Users\carlo\OneDrive\Documents\Modeling and Simulation\CS4632-Electronics-Repair>
```

Shows program can read the csv data and prints out basic information captured from the data.

II: Input Estimate - `python src\print_input_estimates.py`

```
PS C:\Users\carlo\OneDrive\Documents\Modeling and Simulation\CS4632-Electronics-Repair> python src\print_input_estimates.py
RMA request dates
First date: 2024-05-16 00:00:00
Last date: 2026-09-22 00:00:00
Valid date count: 917

Production issue dates
First date: 2024-08-16 00:00:00
Last date: 2026-06-22 00:00:00
Valid date count: 427

Arrival times
RMA arrivals with valid request dates: 917
Average RMA interarrival hours: 22.51
Production arrivals with valid dates: 427
Average production interarrival hours: 38.03

Service times
RMA units with valid service times: 701
Average RMA service hours: 82.55
Production service time assumption: 0.50 hours

Outcomes
repaired: 838 (35.4%)
rtv: 520 (22.0%)
scrap: 181 (7.7%)
unknown: 827 (35.0%)

Capabilities
general: 1564 (66.1%)
specialty: 802 (33.9%)

Priorities
ASAP: 342 (36.2%)
Critical: 125 (13.2%)
Normal: 477 (50.5%)
```

Program can interpret the data and provide estimates based on open/closed RMAs.

Shows dates, service times, outcomes, capabilities, and priorities.

III: Job Creation - `python src\print_created_jobs.py`

```
PS C:\Users\carlo\OneDrive\Documents\Modeling and Simulation\CS4632-Electronics-Repair> python src\print_created_jobs.py
Jobs made from data
Total jobs: 2794
Open RMA rack jobs: 1017
Historical closed RMA jobs: 1192
Open direct request jobs: 91
Historical direct request jobs: 494

Jobs by source
Advance Exchange: 148
Customer RMA: 2209
Production Failure: 428
Reship: 9
```

The CSV data is being converted into jobs. Shows the total created jobs and separates by source.

IV: SimPy and Metrics - python src\print_simpy_run.py

```
PS C:\Users\carlo\OneDrive\Documents\Modeling and Simulation\CS4632-Electronics-Repair> python src\print_simpy_run.py
Open RMA jobs waiting: 1017
Open direct requests waiting: 91

Time 0.00: RMA-1028 pending
Time 0.00: RMA-1052 pending
Time 0.00: RMA-1050 pending
Time 0.00: RMA-1028 started on Tech 2 and Station 2
Time 39.01: RMA-1028 finished
Time 39.01: RMA-1052 started on Tech 2 and Station 2
Time 78.03: RMA-1052 finished
Time 78.03: RMA-1050 started on Tech 2 and Station 2
Time 117.04: RMA-1050 finished

Run results
Jobs finished: 3

Job: RMA-1028
Source: Customer RMA
Capability: specialty
Outcome: rtv
Start time: 0.00
Finish time: 39.01
Wait time: 0.00
Turnaround time: 39.01
Service time: 39.01
Tech used: Tech 2
Station used: Station 2

Job: RMA-1052
Source: Customer RMA
Capability: specialty
Outcome: rtv
Start time: 39.01
Finish time: 78.03
Wait time: 39.01
Turnaround time: 78.03
Service time: 39.01
Tech used: Tech 2
Station used: Station 2

Job: RMA-1050
Source: Customer RMA
Capability: specialty
Outcome: rtv
Start time: 78.03
Finish time: 117.04
Wait time: 78.03
```

First basic SimPy test. Shows jobs are pending in FIFO due to resources. Shows start and finish times. Then prints out basic metrics per job.

Status Summary

Current implementation is basic and has built the foundation for future iterations of development. After M1, I have exported our department's real-world data and cleaned it of any company/customer information for use in the project. I built the basic classes needed to implement the first iteration of simulation. The first simulation runs a test on a small subset of the jobs from the RMA rack and records metrics. Further development will focus on implementing all core features required.

The biggest hurdle thus far has been learning to implement SimPy. It's a new library for me and had to learn how it functioned before implementation. I built the first logic without SimPy for proof of concept before going back and reworking to implement it. The next step in my development is to continue working on the core features required before moving onto running simulations and validation testing.